Assignment 6. jQuery

Team: PixelForge
Members: Marat Nurdaulet; Kadyr Tlektes; Suinalin Azamat
Group: SE-2501
Date: 2 November 2026

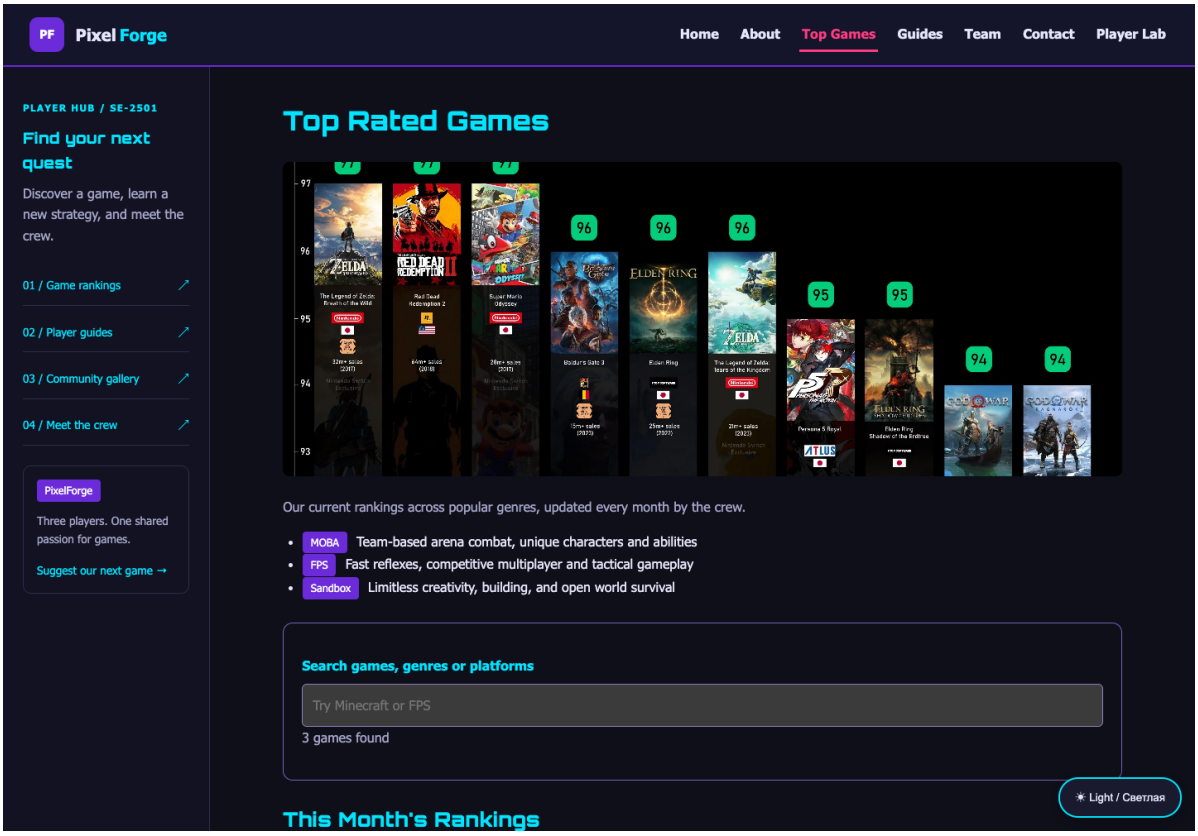
Objective: use jQuery to improve search, engagement and web app functionality.
Continue the existing PixelForge project and implement Tasks 0-9.

Website: https://nurda07.github.io/Web-assignment/games.html
Repository: https://github.com/NURDA07/Web-assignment

Files changed:
assignment3/games.html - required markup and jQuery CDN in the head.
assignment3/script.js - document ready and all required jQuery features.
assignment3/css/jquery.css - styles for the required components.

Run locally: python3 -m http.server 8000 --directory assignment3
Then open http://localhost:8000/games.html.

The form simulates a server call; no message is sent to a server.
The original project pages, assets and previous exercises are preserved.



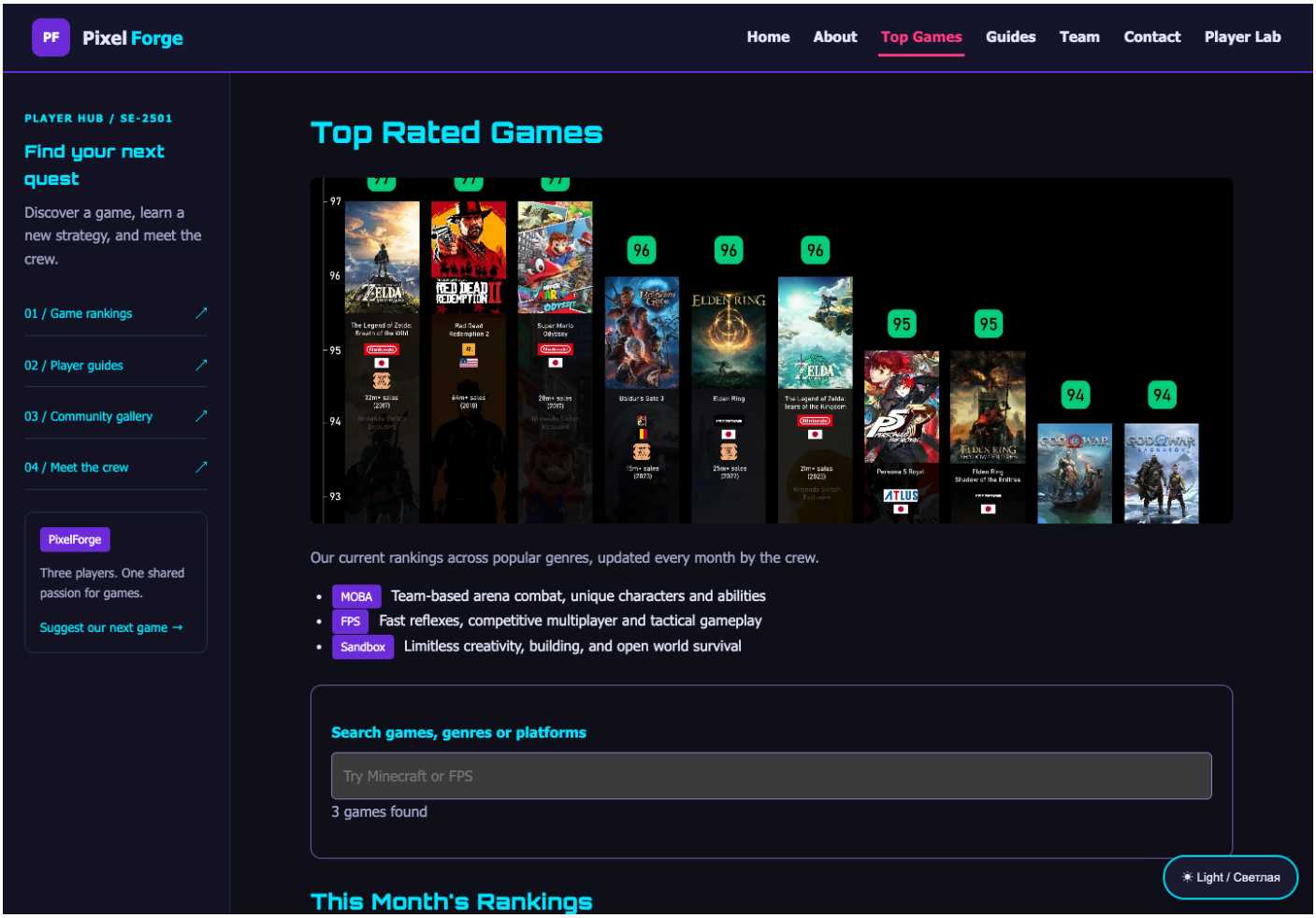
Task 0. Setup

1. Load https://code.jquery.com/jquery-4.0.0.min.js inside the HTML head.
2. Load external script.js immediately after the CDN script.
3. Initialize $(document).ready() and log "jQuery is ready!".

Verified in Chrome: the ready message appears and there are no page errors.

```
<head>
  <script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>
  <script src="script.js"></script>
</head>

$(document).ready(function () {
  console.log("jQuery is ready!");
});
```

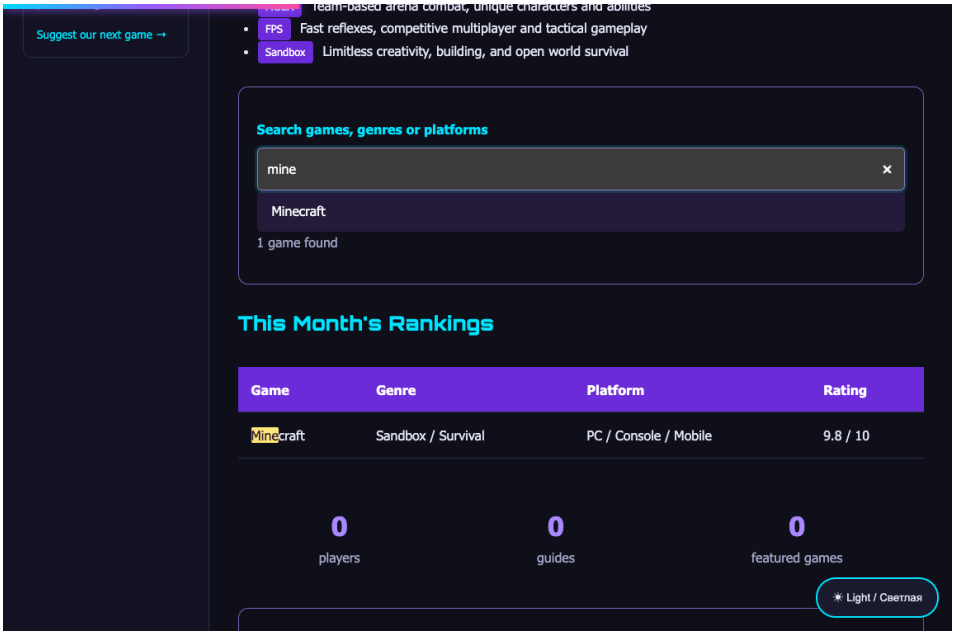


Tasks 1-2. Live search and suggestions

1. Listen to the search input and normalize the keyword to lowercase.
2. Toggle matching ranking rows and update the result count.
3. Filter game names, create suggestion buttons and fill the input on selection.

Verified: "mine" shows only Minecraft; unmatched input shows zero results.
Clearing restores all three rows. Clicking a suggestion fills the search field.

```
47      $("#search-suggestions").prop("hidden", true).empty();
48      $("#game-search").attr("aria-expanded", "false");
49    }
50    $("#game-search").on("input", function () {
51      const query = $(this).val().trim().toLowerCase();
52      clearHighlights();
53      let count = 0;
54      $rows.each(function () {
55        const matches = $(this).text().toLowerCase().includes(query);
56        $(this).toggle(matches);
57        if (matches) count++;
58      });
59      $("#search-status").text(count + (count === 1 ? " game found" : " games found"));
60      closeSuggestions();
61      const suggestions = query ? names.filter(name => name.toLowerCase().includes(query)) : [];
62      suggestions.forEach(function (name) {
63        $("<li>").append($("<button>", {type: "button", text: name})).appendTo("#search-suggestions");
64      });
65      $("#search-suggestions").prop("hidden", !suggestions.length);
66      $(this).attr("aria-expanded", String(!!suggestions.length));
67      highlight(query);
68    }).on("keydown", function (event) {
69      if (event.key === "Escape") closeSuggestions();
70      if (event.key === "ArrowDown" && !$("#search-suggestions").prop("hidden")) {
71        event.preventDefault(); $("#search-suggestions button:first").trigger("focus");
72      }
73    });
74    $("#search-suggestions").on("click", "button", function () {
75      $("#game-search").val($(this).text()).trigger("input").trigger("focus");
76      closeSuggestions();
77    });
78    $(document).on("click", function (event) {
79      if (!$(event.target).closest(".jq-search").length) closeSuggestions();
80    });
81
82    // Tasks 4, 5, 9: scroll progress, counters, and viewport image loading.
83    const counted = new Set();
84    function updateViewport() {
85      const top = $(window).scrollTop();
```



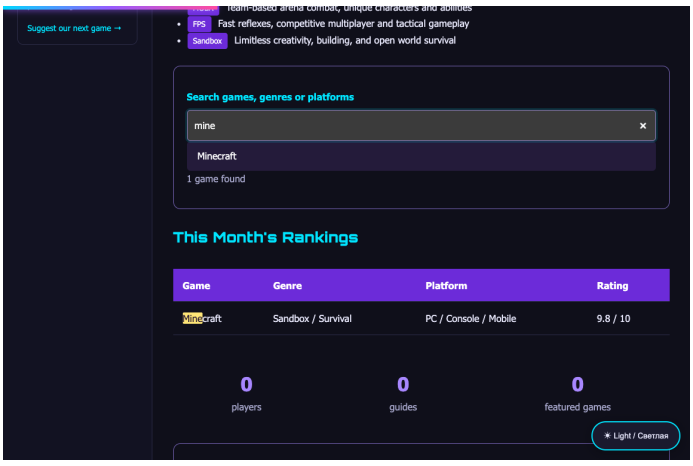
Task 3. Search highlighting

1. Remove earlier marks before processing each new query.
2. Walk text nodes in page content and find every literal matching occurrence.
3. Wrap matches in mark elements while retaining the existing HTML structure.

Verified: matches are highlighted in rankings and gallery text.

Clearing removes the highlights. HTML-like input is treated as text.

```
  4    // Tasks 1-3: live filtering, autocomplete, and safe literal highlighting.
  5    const $rows = $("#game-results tr");
  6    const names = $rows.map(function () { return $(this).find("td:first").text(); }).get();
  7    const searchable = $("main h1, main h2, main p, main li, main td, main figcaption strong, main figcaption
span")
  8      .not("#search-status, #search-suggestions *, .jq-stats *, .jq-panel *");
  9    function clearHighlights() {
 10      $("mark.search-match").each(function () {
 11        const parent = this.parentNode;
 12        $(this).replaceWith(document.createTextNode($(this).text()));
 13        parent.normalize();
 14      });
 15    }
 16    function highlight(query) {
 17      clearHighlights();
 18      if (!query) return;
 19      const seen = new Set();
 20      searchable.each(function () {
 21        const walker = document.createTreeWalker(this, NodeFilter.SHOW_TEXT);
 22        const nodes = [];
 23        while (walker.nextNode()) if (!seen.has(walker.currentNode)) {
 24          seen.add(walker.currentNode); nodes.push(walker.currentNode);
 25        }
 26        nodes.forEach(function (node) {
 27          const value = node.nodeValue;
 28          const lower = value.toLowerCase();
 29          let start = 0, at = lower.indexOf(query);
 30          if (at < 0) return;
 31          const fragment = document.createDocumentFragment();
 32          while (at >= 0) {
 33            fragment.append(document.createTextNode(value.slice(start, at)));
 34            const mark = document.createElement("mark");
 35            mark.className = "search-match";
 36            mark.textContent = value.slice(at, at + query.length);
 37            fragment.append(mark);
 38            start = at + query.length;
 39            at = lower.indexOf(query, start);
 40          }
 41          fragment.append(document.createTextNode(value.slice(start)));
 42          node.replaceWith(fragment);
 43        });
 44      });
 45    }
 46    function closeSuggestions() {
```

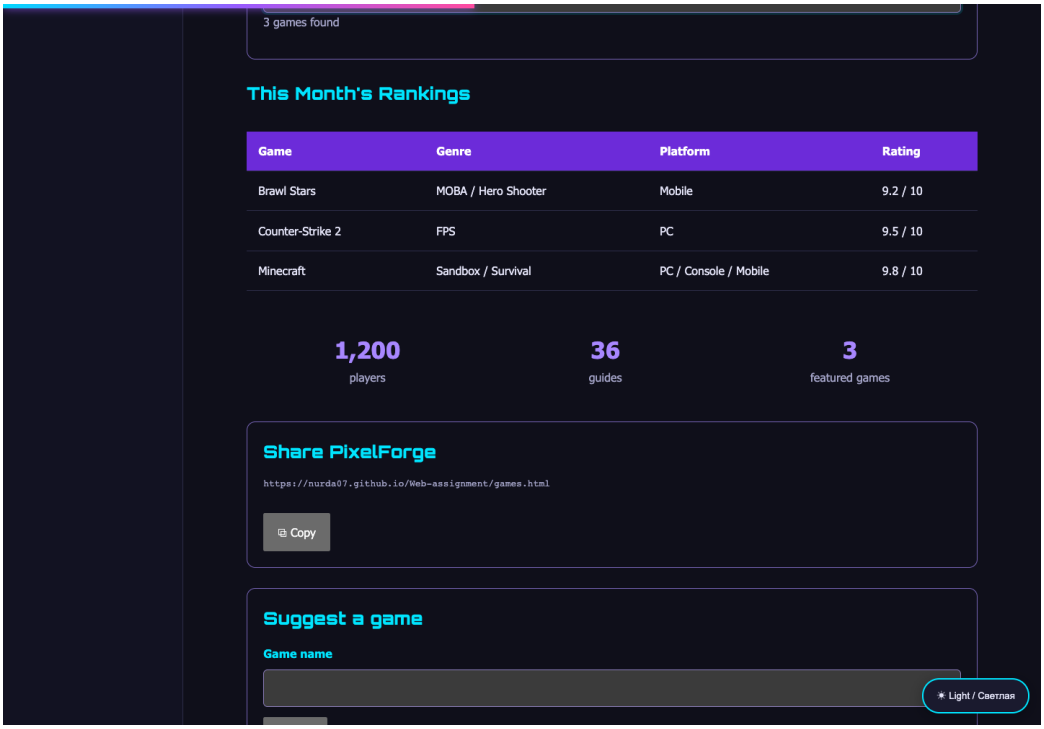


Tasks 4-5. Progress and animated counters

1. On scroll and resize, divide scroll position by the available scroll distance.
2. Set the gradient progress bar width and its accessible percentage value.
3. Start each counter once when its section enters the viewport.
4. Use jQuery animate() to count up smoothly over 1.4 seconds.

Verified: progress reaches 100%; counters finish at 1,200, 36 and 3.

```
82    // Tasks 4, 5, 9: scroll progress, counters, and viewport image loading.
83    const counted = new Set();
84    function updateViewport() {
85      const top = $(window).scrollTop();
86      const bottom = top + $(window).height();
87      const distance = $(document).height() - $(window).height();
88      const progress = distance > 0 ? Math.min(100, top / distance * 100) : 100;
89      $("#scroll-progress").css("width", progress + "%").attr("aria-valuenow", Math.round(progress));
90      $("img[data-src]").each(function () {
91        if ($(this).offset().top < bottom && $(this).offset().top + $(this).height() > top) {
92          $(this).attr("src", $(this).attr("data-src")).removeAttr("data-src");
93        }
94      });
95      $(".number-counter").each(function () {
96        if (counted.has(this) || $(this).offset().top >= bottom) return;
97        counted.add(this);
98        const element = this;
99        $({value: 0}).animate({value: Number($(this).attr("data-count"))}, {
100         duration: 1400, easing: "swing",
101         step: function (value) { $(element).text(Math.floor(value).toLocaleString("en-US")); },
102         complete: function () { $(element).text(Number($(element).attr("data-count")).toLocaleString("en-US"));
}
103       });
104     });
105   }
106   $(window).on("scroll resize", updateViewport);
107   $(".image-gallery img").on("load", updateViewport);
108   updateViewport();
```

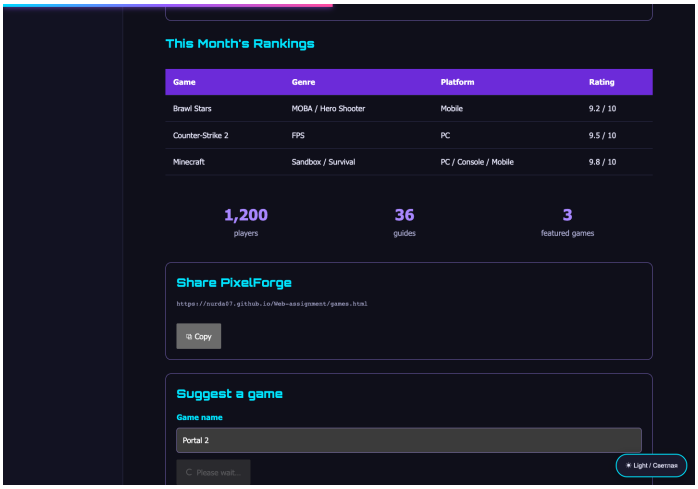


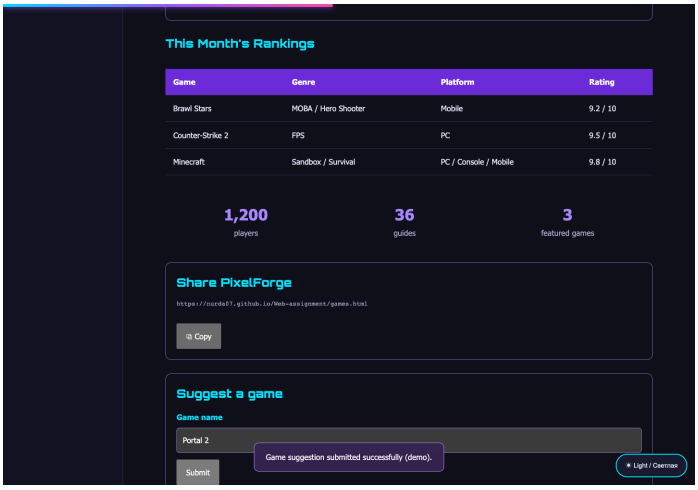
Tasks 6-7. Submit spinner and notifications

1. Handle form submission with jQuery and prevent normal navigation.
2. Disable Submit and show a spinner with "Please wait..." immediately.
3. After 1.6 seconds restore the button and display a demo success toast.
4. Fade the notification out automatically after a short delay.

Verified: the button is disabled while waiting, then enabled; the toast fades.

```
110   // Tasks 6-7: simulated submission, disabled button, and fading notification.
111   let toastTimer;
112   function notify(message) {
113     clearTimeout(toastTimer);
114     $("#notification").stop(true, true).text(message).prop("hidden", false).hide().fadeIn(180);
115     toastTimer = setTimeout(function () {
116       $("#notification").fadeOut(400, function () { $(this).prop("hidden", true); });
117     }, 2400);
118   }
119   $("#game-suggestion-form").on("submit", function (event) {
120     event.preventDefault();
121     const $button = $(this).find("button");
122     if ($button.prop("disabled")) return;
123     $button.prop("disabled", true).html('<span class="spinner" aria-hidden="true"></span>Please wait…');
124     setTimeout(function () {
125       $button.prop("disabled", false).text("Submit");
126       notify("Game suggestion submitted successfully (demo).");
127     }, 1600);
128   });
```



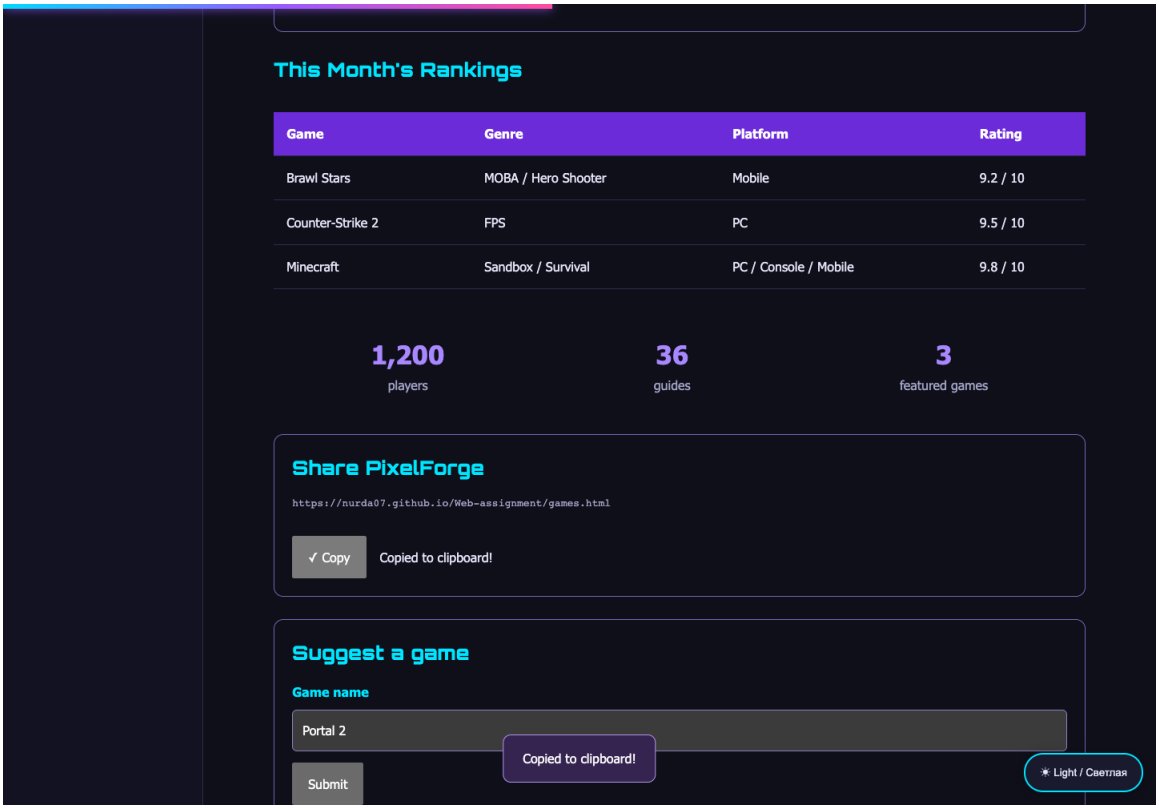


Task 8. Copy to clipboard

1. Handle the Copy button click and write the share URL to the clipboard.
2. After successful copying change the icon to a checkmark.
3. Show "Copied to clipboard!" and restore the button after 2.4 seconds.
4. Display a useful notification if clipboard access fails.

Verified: clipboard content equals the displayed URL; tooltip and check appear.

```
130    // Task 8: copy text; show confirmation only after clipboard success.
131    $("#copy-link").on("click", async function () {
132      const $button = $(this);
133      try {
134        await navigator.clipboard.writeText($("#share-text").text());
135        $button.find(".copy-icon").text("✓");
136        $("#copy-tooltip").prop("hidden", false);
137        notify("Copied to clipboard!");
138        setTimeout(function () {
139          $button.find(".copy-icon").text("⧉");
140          $("#copy-tooltip").prop("hidden", true);
141        }, 2400);
142      } catch (error) { notify("Unable to copy. Select and copy the link manually."); }
143    });
144  });
```

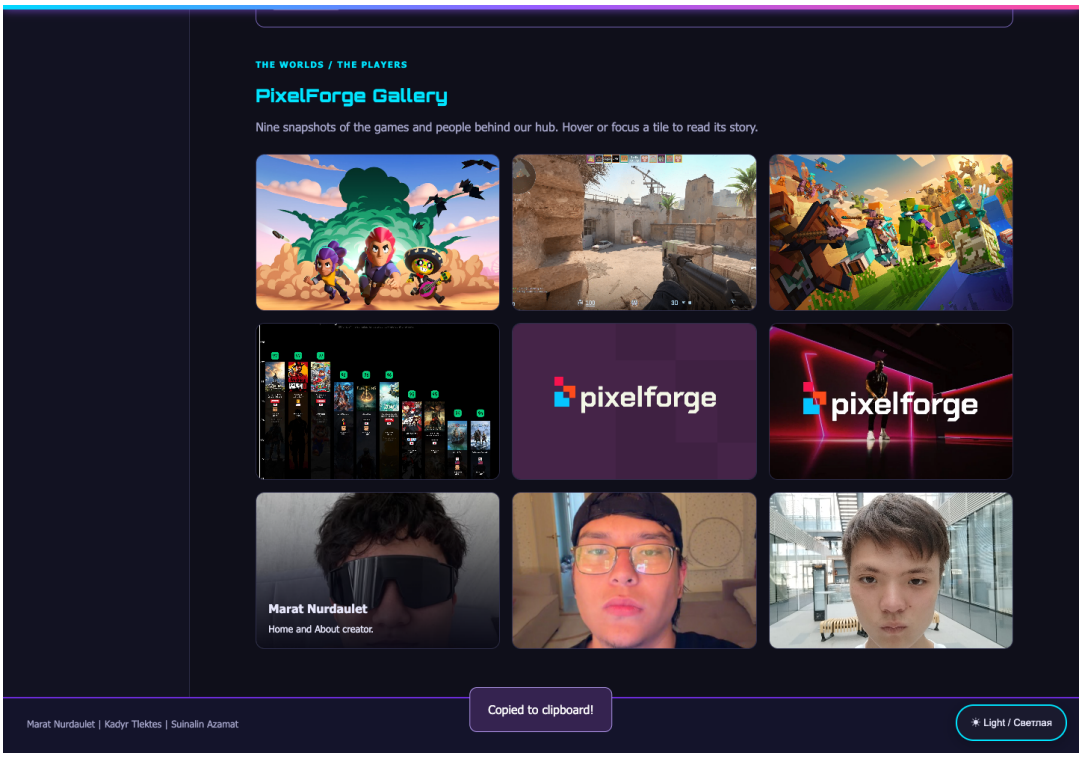


Task 9. Image lazy loading

1. Store gallery image paths in data-src rather than src.
2. Check image positions against the viewport using jQuery offset().
3. Set src only for images entering the viewport, then remove data-src.
4. Recheck on scroll, resize and image load.

Verified: below-screen images initially have no src; all nine load on scrolling.

```
82    // Tasks 4, 5, 9: scroll progress, counters, and viewport image loading.
83    const counted = new Set();
84    function updateViewport() {
85      const top = $(window).scrollTop();
86      const bottom = top + $(window).height();
87      const distance = $(document).height() - $(window).height();
88      const progress = distance > 0 ? Math.min(100, top / distance * 100) : 100;
89      $("#scroll-progress").css("width", progress + "%").attr("aria-valuenow", Math.round(progress));
90      $("img[data-src]").each(function () {
91        if ($(this).offset().top < bottom && $(this).offset().top + $(this).height() > top) {
92          $(this).attr("src", $(this).attr("data-src")).removeAttr("data-src");
93        }
94      });
95      $(".number-counter").each(function () {
96        if (counted.has(this) || $(this).offset().top >= bottom) return;
97        counted.add(this);
98        const element = this;
99        $({value: 0}).animate({value: Number($(this).attr("data-count"))}, {
100         duration: 1400, easing: "swing",
101         step: function (value) { $(element).text(Math.floor(value).toLocaleString("en-US")); },
102         complete: function () { $(element).text(Number($(element).attr("data-count")).toLocaleString("en-US"));
103       }
104       });
105     });
106   }
107   $(window).on("scroll resize", updateViewport);
108   $(".image-gallery img").on("load", updateViewport);
      updateViewport();
```



Deployment, validation and reflection

1. Commit the implementation and push to the shared main branch.
Implementation commit: 5e20aa6.
2. The existing Pages workflow publishes the assignment3 directory.
3. Pages deployment completed successfully; published files match local sources.

Website: https://nurda07.github.io/Web-assignment/games.html
Repository: https://github.com/NURDA07/Web-assignment

Validation: Tasks 0-9 passed automated Chrome checks.
No page JavaScript errors. No horizontal overflow at 390 px.
CDN jQuery 4.0.0 was served from an identical downloaded file during local QA.

Reflection:
Highlighting must preserve nested page elements and treat input literally.
Text-node replacement prevents markup damage and handles special characters.
Lazy images reserve their dimensions to reduce changes to page layout.
Clipboard success is shown only after the asynchronous operation completes.
The form uses a delay to demonstrate loading without requiring a backend.

